

Switching Production Readiness Scenario Checklist

Detailed pre-production test matrix for UAT-to-Production sign-off

Generated: 2026-07-15 03:46:04Z

Scope	Value
Main sections	10
Minimum sub-scenarios	86
Recommended decision	Production sign-off only after every mandatory scenario is PASS or has an approved risk exception.
Primary evidence root	runtime-evidence/prod-readiness-YYYYMMDD-HHMMSS/

- This document defines what should be tested before production go-live.
- Each scenario should produce machine-readable evidence, terminal/API output formatted with jq, and a human-readable RESULT.md.
- A scenario is PASS only when the expected status, state transition, and audit/evidence files are present.

Executive Summary

The production readiness gate is organized into 10 main sections and 86 minimum sub-scenarios. These scenarios cover deployment identity, authentication, payment flows, retries, refunds, DRS, settlement, outbox recovery, configuration hygiene, and observability.

Section	Area	Sub-scenarios	Gate intent
01	Deployment State	8	Prove that the exact release deployed to UAT/production candidate is identifiable, healthy, reproducible, and reversible.
02	Auth Security	10	Prove mTLS, JWT, credentials, roles, and protected admin endpoints are enforced.
03	Payment Happy Path	9	Prove a normal payment can move from inquiry to transfer creation, route selection, dispatch, and success state.
04	Payment Failure Retry	8	Prove retry and failure handling work when a connector rejects, times out, or is unavailable.
05	Refund Reversal	9	Prove post-settlement dispute, maker-checker approval, and refund/reversal completion are reliable.
06	DRS Matrix	8	Prove DRS status matrix, SLA behavior, evidence, and manual escalation paths work.
07	Settlement	10	Prove settlement cycle, DNS/STGS/RTGS style files, callbacks, positions, and reports are consistent.
08	Outbox Recovery	8	Prove outbox reliability, stuck event detection, retry/replay, and health recovery.
09	Config Hardcode Check	7	Prove production-critical settings are externalized and secrets are not hardcoded in source.
10	Observability	9	Prove operators can detect, diagnose, and audit production issues quickly.

Evidence Folder Standard

Use one immutable folder for every full production readiness run. The recommended structure is:

```
runtime-evidence/prod-readiness-YYYYMMDD-HHMMSS/  
00-manifest.json  
01-deployment-state/  
02-auth-security/  
03-payment-happy-path/  
04-payment-failure-retry/  
05-refund-reversal/  
06-drs-matrix/  
07-settlement/  
08-outbox-recovery/  
09-config-hardcode-check/  
10-observability/  
11-score.json  
SUMMARY.md  
SHA256SUMS
```

- Every API response should be stored as .json or .txt with HTTP status when useful.
- Every section should contain a RESULT.md with Status, Evidence, Conclusion, and Known Risks.
- A final SHA256SUMS file should cover every evidence file except SHA256SUMS itself.
- Never store private keys, passwords, raw bearer tokens, or production secrets in evidence.

01 - Deployment State

Objective: Prove that the exact release deployed to UAT/production candidate is identifiable, healthy, reproducible, and reversible.

Minimum scenarios: 8

ID	Scenario	Evidence to collect	Pass criteria
01.1	Build artifact identity	Capture jar checksum, image digest, build timestamp, deployed version.	Artifact checksum matches deployed container.
01.2	Container state	Capture docker compose ps, image tag, restart time, health state.	All required containers are running or healthy.
01.3	Health endpoints	Call /actuator/health with mTLS.	Health returns UP with liveness/readiness.
01.4	Database migration state	Capture Flyway schema history and latest migration.	No failed migration; expected latest version present.
01.5	Configuration profile	Capture active Spring profiles and UAT env keys without secrets.	UAT/prod-like profile is active; secrets not printed.
01.6	Dependency state	Capture Postgres, MinIO, Redpanda readiness.	Dependencies healthy and reachable by app.
01.7	Rollback readiness	Verify backup jar/image exists and deploy script can back up current artifact.	Rollback artifact exists and is documented.
01.8	Clock/timezone sanity	Capture server UTC/local time and app timestamps.	No large drift; timestamps consistent.

02 - Auth Security

Objective: Prove mTLS, JWT, credentials, roles, and protected admin endpoints are enforced.

Minimum scenarios: 10

ID	Scenario	Evidence to collect	Pass criteria
02.1	mTLS required	Call protected endpoint with no client cert.	Rejected at edge with no client certificate.
02.2	mTLS with valid cert	Call health using valid client cert/key/CA.	TLS succeeds.
02.3	No JWT on protected API	Call /api/operations/health with mTLS but no JWT.	401 returned.
02.4	Invalid JWT	Call with Bearer invalid-token.	401 returned.
02.5	Valid admin login	POST /api/auth/login with admin credentials.	Access token returned.
02.6	Wrong password rejection	Login with wrong password.	401 returned.
02.7	Admin endpoint authorization	Call /api/admin/users with admin JWT.	200 returned.
02.8	Maker-checker separation	Try creator self-approval on promotion/push policy.	400 business rejection.
02.9	Session/token expiry posture	Record token expiresIn and refresh behavior if used.	Expiry configured; refresh path controlled.
02.10	Audit security events	Verify auth/admin changes appear in audit logs where supported.	Audit entries present or gap recorded.

03 - Payment Happy Path

Objective: Prove a normal payment can move from inquiry to transfer creation, route selection, dispatch, and success state.

Minimum scenarios: 9

ID	Scenario	Evidence to collect	Pass criteria
03.1	Participant readiness	Verify source and destination participants ACTIVE.	Both ACTIVE.
03.2	Certificate readiness	Verify mTLS certificate inventory for involved banks.	Valid non-expired certs.
03.3	Connector readiness	Test destination connector.	Reachable/healthy or mock SUCCESS.
03.4	Route resolve	Resolve source->destination PACS_008 route.	Expected connector returned.
03.5	Inquiry create/accept	Run inquiry flow for beneficiary/account.	Inquiry accepted and traceable.
03.6	Transfer create	Submit transfer referencing valid inquiry.	Transfer reference returned.
03.7	Outbox dispatch	Verify transfer dispatch event queued and processed.	Outbox SUCCESS.
03.8	Transfer final state	Read transfer by reference.	Status SUCCESS/SETTLEMENT_READY/SETTLED as designed.
03.9	Trace/timeline	Fetch transfer trace/events.	Contains routing, dispatch, response, and state changes.

04 - Payment Failure Retry

Objective: Prove retry and failure handling work when a connector rejects, times out, or is unavailable.

Minimum scenarios: 8

ID	Scenario	Evidence to collect	Pass criteria
04.1	Force reject on connector	Enable forceReject on mock connector.	Connector config updates.
04.2	Submit failing payment	Send payment through force reject route.	Transfer/outbox records failure.
04.3	Retry policy active	Verify TRANSFER push policy active.	Expected retryCount/timeout active.
04.4	Automatic retry observed	Wait or poll retry/outbox history.	Retry attempts increase or retry event recorded.
04.5	Recover connector	Disable forceReject / set SUCCESS mode.	Connector healthy.
04.6	Retry recovery	Manual or automatic retry succeeds.	Outbox SUCCESS and no failed outbox remains for test item.
04.7	Dead-letter behavior	Verify terminal failures appear in dead-letter if retry exhausted.	Dead-letter list accurate or empty after recovery.
04.8	Operations health	Call /api/operations/health after test.	HEALTHY or expected DEGRADED explained.

05 - Refund Reversal

Objective: Prove post-settlement dispute, maker-checker approval, and refund/reversal completion are reliable.

Minimum scenarios: 9

ID	Scenario	Evidence to collect	Pass criteria
05.1	Find settled transfer	Select original settled transfer reference.	Transfer is SETTLED.
05.2	Create post-settlement dispute	POST dispute for settled transfer.	Dispute OPEN.
05.3	Maker submit resolution	Submit REFUND_REQUIRED.	Status PENDING_APPROVAL.
05.4	Checker login	Login checker user.	Token returned.
05.5	Checker approval	Approve resolution.	Status RESOLVED_REFUND.
05.6	Refund record	Fetch dispute detail.	Refund id/ref present.
05.7	Refund completion	Verify refund status.	COMPLETED.
05.8	Transfer markers	Verify original transfer confirmation/settlement confidence.	Marked DISPUTED as designed.
05.9	Health after refund	Call operations health.	HEALTHY or no unresolved outbox failures.

06 - DRS Matrix

Objective: Prove DRS status matrix, SLA behavior, evidence, and manual escalation paths work.

Minimum scenarios: 8

ID	Scenario	Evidence to collect	Pass criteria
06.1	DRS dashboard/list	List DRS disputes.	Endpoint returns expected schema.
06.2	Technical error scenario	Create/identify technical error dispute.	Correct type/status.
06.3	Destination dispute scenario	Create destination/post-settlement dispute.	Correct type/status.
06.4	SLA deadline	Verify SLA deadline exists and is reasonable.	Deadline populated.
06.5	Maker-checker DRS	Submit and approve DRS resolution.	Checker controls enforced.
06.6	Evidence report	Download evidence report/CSV where supported.	Evidence is complete.
06.7	Scheduler behavior	Verify overdue non-settled technical errors escalate, not refund.	Bad auto-refund count zero.
06.8	DRS audit trail	Fetch timeline/audit.	Resolution actions recorded.

07 - Settlement

Objective: Prove settlement cycle, DNS/STGS/RTGS style files, callbacks, positions, and reports are consistent.

Minimum scenarios: 10

ID	Scenario	Evidence to collect	Pass criteria
07.1	Settlement dashboard	Fetch settlement dashboard/list.	Endpoint returns cycles/positions.
07.2	Create/close cycle	Use existing or create settlement cycle.	Cycle closed successfully.
07.3	Generate settlement positions	Fetch cycle positions.	Debit/credit positions balanced.
07.4	Generate instruction/file	Generate/upload payment instruction if supported.	Instruction reference/file present.
07.5	RTGS/STGS callback success	Submit or capture settlement callback.	Cycle/instruction marked settled.
07.6	Settlement reports	Fetch CAMT005/CAMT006/CAMT054 reports.	Reports generated and downloadable.
07.7	No double settlement	Retry callback or duplicate action.	Idempotent; no duplicate ledger movement.
07.8	Failed callback path	Run failed settlement callback scenario.	Failure recorded and recoverable.
07.9	Liquidity/positions view	Verify participant positions.	Amounts match settlement report.
07.10	Settlement audit	Capture audit/timeline for cycle.	Maker/checker and settlement events recorded.

08 - Outbox Recovery

Objective: Prove outbox reliability, stuck event detection, retry/replay, and health recovery.

Minimum scenarios: 8

ID	Scenario	Evidence to collect	Pass criteria
08.1	Operations health before	Capture /api/operations/health.	Baseline recorded.
08.2	Outbox failures list	Fetch failed outbox events.	Failures listed or zero.
08.3	Dead letters list	Fetch /v1/operations/dead-letters.	Dead letters listed or zero.
08.4	Manual retry single	Retry one failed/stuck event if present.	Event moves to SUCCESS or expected terminal state.
08.5	Retry all/recover all	Run recover-all where safe.	Failed count reduced.
08.6	Replay maker-checker	Request/approve/execute replay for dead-letter where applicable.	Maker-checker enforced.
08.7	Outbox health after	Capture operations health again.	HEALTHY or explained residual failures.
08.8	No duplicate side effects	Verify retried event idempotency.	No duplicate transfer/settlement record.

09 - Config Hardcode Check

Objective: Prove production-critical settings are externalized and secrets are not hardcoded in source.

Minimum scenarios: 7

ID	Scenario	Evidence to collect	Pass criteria
09.1	Java source scan	Scan src/main/java for hostnames, passwords, keys, buckets.	No prod secrets hardcoded.
09.2	Resource scan	Scan application.yml/resources.	Only safe local defaults; prod externalized.
09.3	Docker compose env	Review compose env mapping.	Critical settings read from env.
09.4	UAT override check	Capture /opt/switching/docker-compose.override.yml without secrets.	Required env vars present.
09.5	Certificate/key paths	Verify CA key/cert path config comes from env.	No source hardcoding.
09.6	Secret exposure check	Ensure evidence does not include private keys/password dumps.	Secrets redacted.
09.7	Config change principle	Validate change can be made in .env/config without source edit.	Documented and tested.

10 - Observability

Objective: Prove operators can detect, diagnose, and audit production issues quickly.

Minimum scenarios: 9

ID	Scenario	Evidence to collect	Pass criteria
10.1	Health endpoints	Capture actuator and operations health.	Both usable.
10.2	Dashboard summary	Capture operations/dashboard endpoints.	Summary returns current counts.
10.3	Application logs	Capture recent app logs around tests.	Errors understood; no unexplained fatal errors.
10.4	Container logs/state	Capture docker ps/log tail.	Services stable.
10.5	Audit logs	Fetch audit entries for admin changes.	Promotion/policy/connector events visible.
10.6	Metrics endpoint	Capture Prometheus/actuator metrics if exposed.	Metrics scrape works.
10.7	Error correlation	Verify requestId from API error appears in logs.	RequestId searchable.
10.8	Scheduler verification	Verify key schedulers do not generate bad events.	No unexpected scheduler errors.
10.9	Evidence package integrity	Generate SHA256SUMS and summary.	Evidence immutable enough for review.

Production Sign-off Decision Rules

- Mandatory PASS target: 86/86 scenarios, unless a formal risk exception is approved.
- No open P0/P1 security, data integrity, settlement, or retry defects.
- No unexplained 500 errors in the final evidence run.
- All production secrets and environment-specific values must be externalized.
- All core money-moving workflows must have audit trail and replay/retry evidence.
- All evidence folders must include SUMMARY.md, RESULT.md files, and SHA256SUMS.

Recommended score interpretation:

Score	Decision
95-100	Ready for production sign-off if no critical risks remain.
85-94	Ready for continued UAT or limited pilot; production requires exception review.
70-84	Not production ready; resolve material gaps first.
< 70	Blocked; core controls are incomplete.